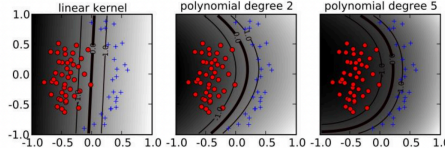


Kernel Types: Polynomial

Fix degree d and constant c :

$$k(x, x') = (x^T x' + c)^d$$

What is $\phi(x)$? Expand the above expression:

$$k(x, x') = (x_1 x'_1 + x_2 x'_2 + c)^2 = \begin{pmatrix} x_1^2 \\ x_2^2 \\ \sqrt{2}x_1 x_2 \\ \sqrt{2}c x_1 \\ \sqrt{2}c x_2 \\ c \end{pmatrix} \cdot \begin{pmatrix} x'_1{}^2 \\ x'_2{}^2 \\ \sqrt{2}x'_1 x'_2 \\ \sqrt{2}c x'_1 \\ \sqrt{2}c x'_2 \\ c \end{pmatrix}$$


Ben-Hur & Weston, Methods in Molecular Biology 2010

what is a general formula for kernel i svm?

In SVMs, a **kernel** is a function

$$k(x, x')$$

that computes an **inner product** in some (possibly very high-dimensional) **feature space** without ever computing the feature map $\phi(x)$ explicitly:

$$k(x, x') = \langle \phi(x), \phi(x') \rangle.$$

This is the **general definition** of a kernel in SVMs.

✓ General Formula for a Kernel in SVM

A function $k : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ is a valid SVM kernel if it satisfies:

Mercer's Condition

The kernel must produce a **positive semi-definite (PSD)** kernel matrix for any dataset.

Formally:

$$k(x, x') = \langle \phi(x), \phi(x') \rangle$$

for some feature map $\phi(\cdot)$.

There is **no single closed-form "general kernel"** because different kernels correspond to very different feature maps. But the definition above is the universal one.

✓ Most Common Kernel Forms in SVMs

1. Linear Kernel

$$k(x, x') = x^\top x'$$

Feature space = original space.

2. Polynomial Kernel

$$k(x, x') = (x^\top x' + c)^d$$

Your slide shows the degree-2 case expanded into its $\phi(x)$.

3. RBF / Gaussian Kernel

Most widely used.

$$k(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2}\right)$$

Corresponds to infinite-dimensional features.

4. Sigmoid / Neural Network Kernel

$$k(x, x') = \tanh(\alpha x^\top x' + c)$$

(Not always PSD, so not always a valid SVM kernel.)

5. General Dot-Product Kernel Family

Any function of inner products:

$$k(x, x') = f(x^\top x')$$

with conditions ensuring PSD.

Summary

The **general formula** of a kernel in SVM is:

$$k(x, x') = \langle \phi(x), \phi(x') \rangle$$

but we evaluate it **without computing** ϕ , using a kernel trick.

For polynomial kernels:

$$k(x, x') = (x^\top x' + c)^d$$

For RBF:

$$k(x, x') = \exp(-\gamma \|x - x'\|^2).$$

If you want, I can:

- ✓ derive $\phi(x)$ for general polynomial degree d ,
 - ✓ compare kernels,
 - ✓ show how kernels warp the feature space,
- or
- ✓ explain how they affect SVM decision boundaries like in your slide.



what is the dual solution for lagrangeian multipliers
alpha_i

Here is **the full dual solution for SVM**, showing exactly where the **Lagrange multipliers** α_i come from and the final **dual optimization problem** they solve.

✓ Primal SVM Optimization Problem

For the hard-margin case:

$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2} \|w\|^2 \\ \text{s.t.} \quad & y_i(w^\top x_i + b) \geq 1, \quad i = 1, \dots, n \end{aligned}$$

To solve this, we introduce **Lagrange multipliers** $\alpha_i \geq 0$.

✓ Lagrangian

$$L(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_{i=1}^n \alpha_i [y_i(w^\top x_i + b) - 1]$$

✓ Dual Derivation

1. Set derivative w.r.t. w to zero:

$$\frac{\partial L}{\partial w} = w - \sum_{i=1}^n \alpha_i y_i x_i = 0$$

so

$$w = \sum_{i=1}^n \alpha_i y_i x_i$$

This is a key result:

The weight vector is a weighted sum of training points.

Only support vectors have $\alpha_i > 0$.

2. Set derivative w.r.t. b to zero:

$$\frac{\partial L}{\partial b} = - \sum_{i=1}^n \alpha_i y_i = 0$$

so

$$\sum_{i=1}^n \alpha_i y_i = 0$$

3. Plug critical values back into the Lagrangian → Dual problem.

The dual becomes:

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i^\top x_j) \\ \text{s.t.} \quad & \sum_{i=1}^n \alpha_i y_i = 0 \\ & \alpha_i \geq 0 \end{aligned}$$

This is the **dual SVM optimization problem**.

★ Final Dual Solution With Kernel Trick

Replace $x_i^\top x_j$ by a kernel:

$$k(x_i, x_j) = \phi(x_i)^\top \phi(x_j)$$

Then the dual becomes:

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j k(x_i, x_j) \\ \text{s.t.} \quad & \sum_{i=1}^n \alpha_i y_i = 0 \\ & \alpha_i \geq 0 \end{aligned}$$

This is the final dual SVM objective you solve numerically.

🧩 Decision Function (Classification Rule)

Once α_i are found:

$$w = \sum_{i=1}^n \alpha_i y_i x_i$$

Prediction for new point x :

$$f(x) = \sum_{i=1}^n \alpha_i y_i k(x_i, x) + b$$

Support vectors are exactly the points with:

$$\alpha_i > 0$$

Summary: Dual Solution for α_i

$$\alpha^* = \arg \max_{\alpha} \left(\sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j k(x_i, x_j) \right)$$

Subject to:

$$\alpha_i \geq 0, \quad \sum_i \alpha_i y_i = 0$$

If you'd like, I can also:

- ✓ derive the **soft-margin** dual (with $0 \leq \alpha_i \leq C$)
 - ✓ show how KKT conditions determine which points are support vectors
 - ✓ draw a diagram of what α_i represents
- or
- ✓ explain dual vs primal intuition.

Just tell me!



can you give me the formulas for gaussian kernel

Here are **all the standard formulas for the Gaussian (RBF) kernel**, including the kernel itself, the distance form, the similarity form, and the feature-space intuition.

✓ 1. Standard Gaussian (RBF) Kernel

The most common form:

$$k(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2}\right)$$

- σ = bandwidth (controls smoothness)
 - $\|x - x'\|^2$ = Euclidean distance squared
-

✓ 2. Equivalent Form Using γ

Many libraries (SVM in scikit-learn, libSVM, etc.) use:

$$k(x, x') = \exp(-\gamma\|x - x'\|^2)$$

with the relation:

$$\gamma = \frac{1}{2\sigma^2}$$

✓ 3. Expanded Distance Expression

Since

$$\|x - x'\|^2 = x^\top x + x'^\top x' - 2x^\top x',$$

then:

$$k(x, x') = \exp(-\gamma(x^\top x + x'^\top x' - 2x^\top x'))$$

This is useful to see how the kernel relates to inner products.

✓ 4. Feature-Space Interpretation (Infinite Dimensions)

There exists a map $\phi(x)$ such that:

$$k(x, x') = \langle \phi(x), \phi(x') \rangle$$

and $\phi(x)$ has **infinitely many terms**:

$$\phi(x) = [1, x, x^2, x^3, x^4, \dots]$$

in a weighted Gaussian basis.

You never compute $\phi(x)$ directly—this is the **kernel trick**.

✓ 5. Gaussian Kernel in 1D (example)

$$k(x, x') = e^{-\gamma(x-x')^2}$$

Peak at 1 when $x = x'$, quickly decays to 0 as distance grows.

★ Summary (Most Important)

$$k(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2}\right) = \exp(-\gamma\|x - x'\|^2)$$

with

$$\gamma = \frac{1}{2\sigma^2}.$$

If you'd like, I can also show:

- ✓ how Gaussian kernels warp the decision boundary,
 - ✓ the effect of γ on overfitting/underfitting,
- or
- ✓ how Gaussian kernels relate to radial basis functions mathematically.



↪ $\max_{\alpha} s.t. \sum_{i=1}^n \alpha_i - 2\alpha_l = 1, \sum_{j=1}^n \alpha_j = 1, \sum_{i=1}^n \alpha_i y_i y_j k(x_i, x_j) = 1, \sum_{i=1}^n \alpha_i y_i = 0, \alpha_i \geq 0$

can you give me the formulas for gaussian kernel for this formula

Got it — you want the **Gaussian (RBF) kernel inserted directly into the SVM dual formula:**

$$\max_{\alpha} \quad \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j k(x_i, x_j)$$

with the constraints:

$$\sum_{i=1}^n \alpha_i y_i = 0, \quad \alpha_i \geq 0.$$

Below is the **exact formula** when you plug in the Gaussian/RBF kernel.

✓ Gaussian Kernel Formula

$$k(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

where

$$\gamma = \frac{1}{2\sigma^2}.$$

Or equivalently:

$$k(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$$

✓ Plugging Gaussian Kernel Into the SVM Dual

Dual Optimization with Gaussian Kernel

$$\max_{\alpha} \quad \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \exp(-\gamma \|x_i - x_j\|^2)$$

subject to:

$$\sum_{i=1}^n \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C$$

(If hard-margin, remove C : then $\alpha_i \geq 0$.)

✓ Explicit Expanded Distance Form

Since

$$\|x_i - x_j\|^2 = x_i^\top x_i + x_j^\top x_j - 2x_i^\top x_j,$$

the kernel can be written as:

$$k(x_i, x_j) = \exp(-\gamma(x_i^\top x_i + x_j^\top x_j - 2x_i^\top x_j)).$$

So the dual becomes:

$$\max_{\alpha} \quad \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j \exp(-\gamma(x_i^\top x_i + x_j^\top x_j - 2x_i^\top x_j)).$$

★ Final SVM Dual With Gaussian Kernel (clean form)

$$\max_{\alpha} \quad \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j \exp(-\gamma \|x_i - x_j\|^2)$$

$$\text{s.t.} \quad \sum_i \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C.$$

If you want, I can also derive:

- ✓ the optimal $\mathbf{b}$ formula when using Gaussian kernels
- ✓ how support vectors are chosen
- ✓ how the decision function becomes

$$f(x) = \sum_i \alpha_i y_i k(x_i, x) + b$$

Just tell me!



i want w and b calculated by the system above

Thought for 6s >

Nice, this is the key step after solving for the α_i 's.

We start from the dual (with any kernel, including Gaussian):

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j k(x_i, x_j) \quad \text{s.t.} \quad \sum_i \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C$$

Let α_i^* be the optimal solution.

1. w in terms of α_i

In feature space $\phi(x)$:

$$w = \sum_{i=1}^n \alpha_i^* y_i \phi(x_i)$$

- For a **linear kernel** $k(x_i, x_j) = x_i^\top x_j$, $\phi(x) = x$, so

$$w = \sum_{i=1}^n \alpha_i^* y_i x_i$$

- For a **Gaussian kernel**, $\phi(x)$ is infinite-dimensional, so we **do not explicitly write** w ; we only use it through kernels in the decision function:

$$f(x) = \underbrace{\sum_{i=1}^n \alpha_i^* y_i k(x_i, x)}_{\langle w, \phi(x) \rangle} + b$$

So: with Gaussian kernel you *conceptually* have $w = \sum \alpha_i y_i \phi(x_i)$, but you never materialize it.

2. Computing b from α_i^*

Use the KKT condition on any **support vector** with

$$0 < \alpha_i^* < C:$$

$$y_i (\langle w, \phi(x_i) \rangle + b) = 1$$

Substitute $w = \sum_j \alpha_j^* y_j \phi(x_j)$:

$$1 = y_i \left(\sum_{j=1}^n \alpha_j^* y_j k(x_j, x_i) + b \right)$$

Solve for b :

$$b = y_i - \sum_{j=1}^n \alpha_j^* y_j k(x_j, x_i) \quad \text{for any } i \text{ with } 0 < \alpha_i^* < C$$

In practice you often **average** over all such support vectors:

$$b = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \left(y_i - \sum_{j=1}^n \alpha_j^* y_j k(x_j, x_i) \right), \quad \mathcal{S} = \{i : 0 < \alpha_i^* < C\}$$

Final decision function (Gaussian kernel)

With the Gaussian kernel

$$k(x_j, x) = \exp(-\gamma \|x_j - x\|^2):$$

$$f(x) = \sum_{j=1}^n \alpha_j^* y_j \exp(-\gamma \|x_j - x\|^2) + b$$

and you classify via $\text{sign}(f(x))$.

If you tell me whether you're in the **linear** or **Gaussian** case, I can plug in numbers or give you a tiny worked example.



↳ So: with Gaussian kernel you conceptually have $w = \sum \alpha_i y_i \phi(x_i)$ but you never materialize it.

can you further explain this

Absolutely — this is *the* key conceptual point in kernelized SVMs, and it's what usually confuses people.

Let me break it down clearly and visually.

✓ 1. In a linear SVM, w lives in the input space

If your data is in $\mathbb{R}^d$, then:

$$w = \sum_{i=1}^n \alpha_i y_i x_i \in \mathbb{R}^d$$

So you can actually compute and store w directly — a d -dimensional vector.

✓ 2. In a Gaussian (RBF) SVM, the feature map $\phi(x)$ is infinite-dimensional

The Gaussian kernel:

$$k(x, x') = e^{-\gamma \|x - x'\|^2}$$

corresponds to a feature mapping:

$$\phi(x) = [1, x, x^2, x^3, \dots \text{infinitely many terms} \dots]$$

There is no closed-form finite vector for $\phi(x)$.

It has **infinitely many components**, like a Taylor expansion.

! Why does this matter?

Because the formula for the SVM weight vector is:

$$w = \sum_{i=1}^n \alpha_i y_i \phi(x_i)$$

If $\phi(x_i)$ is infinite-dimensional, then:

► **w is also infinite-dimensional**

You **cannot compute it directly**.

You cannot store it in memory.

You cannot even write it down.

This is why, with Gaussian kernels, **you never explicitly compute w** .

✓ 3. But you *can* compute dot products with w : kernel trick

Even though you cannot compute $\phi(x)$ or w ,
you *can compute* inner products like:

$$\langle w, \phi(x) \rangle$$

because:

$$\langle w, \phi(x) \rangle = \sum_{i=1}^n \alpha_i y_i \langle \phi(x_i), \phi(x) \rangle = \sum_{i=1}^n \alpha_i y_i k(x_i, x)$$

And that is **fully computable**:

$$k(x_i, x) = e^{-\gamma \|x_i - x\|^2}$$

So the classifier is:

$$f(x) = \sum_i \alpha_i y_i e^{-\gamma \|x_i - x\|^2} + b$$

No infinite-dimensional vectors ever appear — only kernels.

4. Intuition: Gaussian kernel makes SVM nonlinear without computing features

Think of Gaussian kernel as placing a “bump” around each support vector:

$$k(x_i, x) \approx \begin{cases} 1 & \text{if } x \approx x_i \\ 0 & \text{if far away} \end{cases}$$

So the decision boundary becomes:

- smooth
- curved
- nonlinear
- shaped by the support vectors

But again:

you never compute w explicitly, because its representation would require storing infinitely many numbers.

5. Summary

✓ Linear SVM

You compute

$$w = \sum \alpha_i y_i x_i \quad (\text{finite vector})$$

✓ Gaussian SVM

You *cannot* compute

$$w = \sum \alpha_i y_i \phi(x_i) \quad (\text{infinite vector})$$

Instead you compute predictions via:

$$f(x) = \sum_i \alpha_i y_i k(x_i, x) + b.$$

This is the **kernel trick** — it avoids computing the infinite-dimensional $\phi(x)$ by computing only kernel values.

If you want:

I can show:

- 📌 a small 2-D example showing how w “would” look
- 📌 how the Gaussian feature space is infinite
- 📌 pictures of nonlinear boundaries using Gaussian kernels
- 📌 how the dual and primal differ in linear vs. kernel SVM

Just tell me!



↪ $f(x)$

is this the separation line?

Yes — the separation boundary in a kernel SVM is exactly the set of points where the decision function equals zero:

$f(x) = 0$

Let me explain in a super clear way.

✓ 1. The SVM decision function

For a Gaussian-kernel SVM:

$$f(x) = \sum_{i=1}^n \alpha_i y_i k(x_i, x) + b$$

This is the function the SVM learns.

- If $f(x) > 0 \rightarrow$ predict +1
 - If $f(x) < 0 \rightarrow$ predict -1
-

🚧 2. The separation boundary (decision boundary)

The **separation line** (in 2D) or **separation surface** (in higher dimensions) is:

$$f(x) = 0$$

This is the curve (or surface) where the classifier switches from +1 to -1.

For Gaussian kernels, it is **not a straight line** — it becomes curved, smooth, and shaped by the support vectors.

🎯 3. The margins

Margins are also given by level sets of $f(x)$:

- Positive margin:

$$f(x) = +1$$

- Negative margin:

$$f(x) = -1$$

Points satisfying

$$y_i f(x_i) = 1$$

are support vectors.

4. Why this is the separation line

In a linear SVM:

$$f(x) = w^\top x + b$$

and the separation line is:

$$w^\top x + b = 0$$

With a kernel SVM:

$$f(x) = \sum_i \alpha_i y_i k(x_i, x) + b$$

This plays the exact same role, just in feature space.

You never see the feature space directly — only the decision boundary implicitly through $f(x) = 0$.

★ Final Answer

✓ Yes —

$f(x) = 0$ is the separation line (decision boundary).

✓ And

$f(x) = \pm 1$ are the margin boundaries.